Panduan Vibecoding: Membuat Aplikasi Kasir dari Nol

Tutorial Step-by-Step dengan Contoh Nyata

Disusun oleh: Siraj Nur Ihrom, Founder & Lead Developer SNISHOP
Versi: 1.0
Tanggal: Oktober 2026
Format: Tutorial lengkap dengan kode, screenshot, dan panduan deployment
Standar: SNISHOP-PUBLISHING v2.0
Profil: Tutorial Step-by-Step

Tentang Buku Ini

Buku ini adalah panduan praktis untuk membuat aplikasi kasir (Point of Sale) dari nol menggunakan pendekatan vibecoding. Anda tidak perlu pengalaman pemrograman sebelumnya. Kami akan membangun aplikasi kasir yang bisa langsung dipakai untuk bisnis Anda.

Yang akan Anda bangun:

- Aplikasi kasir berbasis web yang bisa diakses dari browser
- Fitur produk, transaksi, laporan penjualan, dan manajemen stok
- Database untuk menyimpan data produk dan transaksi
- Antarmuka yang mudah dipakai oleh kasir
- Deployment ke server agar bisa diakses dari mana saja

Catatan untuk peninjau: Screenshot aplikasi kasir final yang sudah jadi diperlukan di bagian ini untuk menunjukkan hasil akhir. [PERLU SCREENSHOT: Tampilan aplikasi kasir final dengan dashboard, halaman produk, dan halaman transaksi]

Prasyarat

Sebelum memulai, pastikan Anda memiliki:

- Laptop atau komputer dengan koneksi internet
- Browser modern (Chrome, Firefox, atau Edge)
- Akun GitHub (gratis) untuk menyimpan kode
- Akun Vercel (gratis) untuk deployment
- Text editor (kami akan pakai Visual Studio Code)

Catatan untuk peninjau: Screenshot halaman pendaftaran GitHub dan Vercel diperlukan di bagian ini. [PERLU SCREENSHOT: Halaman signup GitHub dan Vercel]

Apa Itu Vibecoding?

Vibecoding adalah pendekatan pemrograman yang mengandalkan AI untuk membantu menulis kode. Anda menjelaskan apa yang ingin Anda bangun dalam bahasa manusia, dan AI akan menghasilkan kode untuk Anda. Tugas Anda adalah memahami, memodifikasi, dan menguji kode tersebut.

Vibecoding bukan tentang mengganti programmer. Vibecoding adalah tentang mempercepat pengembangan untuk orang yang punya ide bisnis tapi tidak punya latar belakang teknis.

Kelebihan Vibecoding

1. **Cepat**: Aplikasi yang biasanya butuh berminggu-minggu bisa selesai dalam hitungan jam
2. **Murah**: Tidak perlu menyewa programmer mahal untuk MVP
3. **Fleksibel**: Anda bisa mengubah fitur kapan saja tanpa menunggu developer
4. **Edukatif**: Anda belajar pemrograman sambil membangun produk nyata

Keterbatasan Vibecoding

1. **Kualitas kode**: Kode yang dihasilkan AI tidak selalu optimal atau aman
2. **Debugging**: Ketika ada bug, Anda perlu pemahaman dasar untuk memperbaikinya
3. **Skalabilitas**: Aplikasi vibecoding cocok untuk MVP, tapi mungkin perlu refactor untuk skala besar
4. **Keamanan**: Anda perlu memeriksa aspek keamanan sendiri atau konsultasi dengan ahli

Arsitektur Aplikasi

Aplikasi kasir yang akan kita bangun menggunakan stack berikut:

Komponen	Teknologi	Fungsi
Frontend	React + Tailwind CSS	Antarmuka pengguna
Backend	Node.js + Express	API dan logika bisnis
Database	PostgreSQL	Penyimpanan data
Authentication	JWT	Login dan keamanan
Deployment	Vercel	Hosting aplikasi

Catatan untuk peninjau: Diagram arsitektur aplikasi diperlukan di bagian ini. [PERLU DIAGRAM: Arsitektur aplikasi kasir dengan frontend, backend, database, dan deployment]

Bab 1: Menyiapkan Lingkungan Pengembangan

1.1 Instal Visual Studio Code

Visual Studio Code (VS Code) adalah text editor gratis yang populer di kalangan developer.

Langkah 1: Unduh VS Code dari https://code.visualstudio.com

Langkah 2: Jalankan installer dan ikuti petunjuknya

Langkah 3: Buka VS Code setelah instalasi selesai

Catatan untuk peninjau: Screenshot halaman unduh VS Code dan tampilan awal VS Code diperlukan di bagian ini. [PERLU SCREENSHOT: Halaman unduh VS Code dan tampilan awal setelah instalasi]

1.2 Instal Extension yang Dibutuhkan

VS Code membutuhkan extension untuk mendukung pengembangan web.

Langkah 1: Buka menu Extensions (Ctrl+Shift+X atau Cmd+Shift+X)

Langkah 2: Cari dan instal extension berikut:

- ESLint
- Prettier - Code formatter
- ES7+ React/Redux/React-Native snippets
- Tailwind CSS IntelliSense
- PostgreSQL (oleh Chris Kolk)

Catatan untuk peninjau: Screenshot halaman Extensions VS Code dengan extension yang dicari diperlukan di bagian ini. [PERLU SCREENSHOT: Halaman Extensions VS Code dengan 5 extension yang disebutkan]

1.3 Instal Node.js

Node.js adalah runtime JavaScript yang akan kita pakai untuk backend.

Langkah 1: Unduh Node.js LTS dari https://nodejs.org

Langkah 2: Jalankan installer dan ikuti petunjuknya

Langkah 3: Verifikasi instalasi dengan membuka terminal dan menjalankan:

```
node --version
npm --version
```

Anda harus melihat nomor versi seperti `v20.10.0` dan `10.2.3`.

Catatan untuk peninjau: Screenshot terminal dengan output versi Node.js dan npm diperlukan di bagian ini. [PERLU SCREENSHOT: Terminal menampilkan output `node --version` dan `npm --version`]

1.4 Instal PostgreSQL

PostgreSQL adalah database yang akan kita pakai untuk menyimpan data.

Langkah 1: Unduh PostgreSQL dari https://www.postgresql.org/download

Langkah 2: Jalankan installer dan ikuti petunjuknya

Langkah 3: Saat diminta, set password untuk user `postgres`. Catat password ini.

Langkah 4: Instal pgAdmin (termasuk dalam installer) untuk mengelola database secara visual

Catatan untuk peninjau: Screenshot installer PostgreSQL dan pgAdmin diperlukan di bagian ini. [PERLU SCREENSHOT: Installer PostgreSQL dan tampilan awal pgAdmin]

1.5 Buat Database untuk Aplikasi Kasir

Langkah 1: Buka pgAdmin dan login dengan password yang Anda set

Langkah 2: Klik kanan pada "Databases" dan pilih "Create" > "Database"

Langkah 3: Beri nama database `kasir_db` dan klik "Save"

Langkah 4: Verifikasi database sudah terbuat dengan melihatnya di sidebar kiri

Catatan untuk peninjau: Screenshot pgAdmin dengan database `kasir_db` yang sudah terbuat diperlukan di bagian ini. [PERLU SCREENSHOT: pgAdmin menampilkan database `kasir_db`]

Bab 2: Membangun Backend API

2.1 Inisialisasi Proyek

Langkah 1: Buat folder baru untuk proyek:

```
mkdir kasir-app
cd kasir-app
```

Langkah 2: Inisialisasi proyek Node.js:

```
npm init -y
```

Langkah 3: Instal dependensi yang dibutuhkan:

```
npm install express pg cors dotenv bcryptjs jsonwebtoken
npm install --save-dev nodemon
```

Penjelasan dependensi:

- `express` : Framework web untuk Node.js
- `pg` : Driver PostgreSQL untuk Node.js
- `cors` : Mengizinkan request dari domain berbeda
- `dotenv` : Mengelola environment variables
- `bcryptjs` : Enkripsi password
- `jsonwebtoken` : Authentication token
- `nodemon` : Auto-restart server saat kode berubah (development)

Catatan untuk peninjau: Screenshot terminal dengan output instalasi dependensi diperlukan di bagian ini. [PERLU SCREENSHOT: Terminal menampilkan output `npm install` dengan daftar package yang diinstal]

2.2 Konfigurasi Environment Variables

Langkah 1: Buat file `.env` di root proyek:

```
touch .env
```

Langkah 2: Isi file `.env` dengan konfigurasi berikut:

```
PORT=5000
DATABASE_URL=postgresql://postgres:YOUR_PASSWORD@localhost:5432/kasir_db
JWT_SECRET=your-secret-key-change-this-in-production
```

Ganti `YOUR_PASSWORD` dengan password PostgreSQL yang Anda set sebelumnya.

Langkah 3: Tambahkan `.env` ke `.gitignore` :

```
echo ".env" >> .gitignore
```

Catatan untuk peninjau: Screenshot file `.env` dan `.gitignore` diperlukan di bagian ini. [PERLU SCREENSHOT: Isi file `.env` dan `.gitignore`]

2.3 Koneksi ke Database

Langkah 1: Buat file `db.js` di root proyek:

```
const { Pool } = require('pg');
require('dotenv').config();

const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
});

pool.on('connect', () => {
  console.log('Connected to PostgreSQL database');
});

pool.on('error', (err) => {
  console.error('Unexpected error on idle client', err);
  process.exit(-1);
});

module.exports = pool;
```

Penjelasan: File ini membuat koneksi ke database PostgreSQL menggunakan connection string dari environment variables.

Catatan untuk peninjau: Screenshot file `db.js` di VS Code diperlukan di bagian ini. [PERLU SCREENSHOT: File `db.js` di VS Code dengan syntax highlighting]

2.4 Membuat Tabel Database

Langkah 1: Buat file `init-db.js` untuk membuat tabel:

```
const pool = require('./db');

const createTables = async () => {
  try {
    // Tabel users untuk authentication
    await pool.query(`
      CREATE TABLE IF NOT EXISTS users (
        id SERIAL PRIMARY KEY,
        username VARCHAR(50) UNIQUE NOT NULL,
        password VARCHAR(255) NOT NULL,
        role VARCHAR(20) NOT NULL DEFAULT 'kasir',
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
      );
    `);

    // Tabel produk
    await pool.query(`
      CREATE TABLE IF NOT EXISTS products (
        id SERIAL PRIMARY KEY,
        name VARCHAR(100) NOT NULL,
        price INTEGER NOT NULL,
        stock INTEGER NOT NULL DEFAULT 0,
        category VARCHAR(50),
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
      );
    `);

    // Tabel transaksi
    await pool.query(`
      CREATE TABLE IF NOT EXISTS transactions (
        id SERIAL PRIMARY KEY,
        transaction_number VARCHAR(50) UNIQUE NOT NULL,
        total_amount INTEGER NOT NULL,
        payment_method VARCHAR(20) NOT NULL,
        cashier_id INTEGER REFERENCES users(id),
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
      );
    `);

    // Tabel detail transaksi
    await pool.query(`
      CREATE TABLE IF NOT EXISTS transaction_items (
        id SERIAL PRIMARY KEY,
        transaction_id INTEGER REFERENCES transactions(id) ON DELETE CASCADE,
        product_id INTEGER REFERENCES products(id),
        quantity INTEGER NOT NULL,
        price INTEGER NOT NULL,
        subtotal INTEGER NOT NULL
      );
    `);

    console.log('All tables created successfully');
  } catch (err) {
    console.error('Error creating tables:', err);
```

```
  }
};

createTables();
```

Langkah 2: Jalankan script untuk membuat tabel:

```
node init-db.js
```

Anda harus melihat pesan "All tables created successfully".

Langkah 3: Verifikasi tabel sudah terbuat di pgAdmin

Catatan untuk peninjau: Screenshot pgAdmin dengan 4 tabel yang sudah terbuat diperlukan di bagian ini. [PERLU SCREENSHOT: pgAdmin menampilkan tabel users, products, transactions, dan transaction_items]

2.5 Membuat Server Express

Langkah 1: Buat file `server.js`:

```
const express = require('express');
const cors = require('cors');
require('dotenv').config();

const app = express();
const PORT = process.env.PORT || 5000;

// Middleware
app.use(cors());
app.use(express.json());

// Routes
app.get('/', (req, res) => {
  res.json({ message: 'Kasir API is running' });
});

// Start server
app.listen(PORT, () => {
  console.log(`Server is running on port ${PORT}`);
});
```

Langkah 2: Tambahkan script di `package.json`:

```
{
  "scripts": {
    "dev": "nodemon server.js",
    "start": "node server.js"
  }
}
```

Langkah 3: Jalankan server dalam mode development:

```
npm run dev
```

Anda harus melihat pesan "Server is running on port 5000".

Langkah 4: Buka browser dan akses http://localhost:5000. Anda harus melihat pesan JSON `{"message":"Kasir API is running"}`.

Catatan untuk peninjau: Screenshot terminal dengan pesan server berjalan dan browser dengan response JSON diperlukan di bagian ini. [PERLU SCREENSHOT: Terminal dengan "Server is running on port 5000" dan browser dengan response JSON]

Bab 3: Implementasi Authentication

3.1 Endpoint Registrasi User

Langkah 1: Buat folder `routes` dan file `auth.js`:

```
mkdir routes
touch routes/auth.js
```

Langkah 2: Isi file `routes/auth.js`:

```
const express = require('express');
const router = express.Router();
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const pool = require('../db');

// Registrasi user
router.post('/register', async (req, res) => {
  try {
    const { username, password, role } = req.body;

    // Validasi input
    if (!username || !password) {
      return res.status(400).json({ error: 'Username dan password wajib diisi' });
    }

    // Cek apakah username sudah ada
    const existingUser = await pool.query(
      'SELECT * FROM users WHERE username = $1',
      [username]
    );

    if (existingUser.rows.length > 0) {
      return res.status(400).json({ error: 'Username sudah digunakan' });
    }

    // Enkripsi password
    const hashedPassword = await bcrypt.hash(password, 10);

    // Simpan user ke database
    const newUser = await pool.query(
        'INSERT INTO users (username, password, role) VALUES ($1, $2, $3) RETURNING id,
username, role',
      [username, hashedPassword, role || 'kasir']
    );

    res.status(201).json({
      message: 'User berhasil didaftarkan',
      user: newUser.rows[0]
    });
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'Server error' });
  }
});

// Login user
router.post('/login', async (req, res) => {
  try {
    const { username, password } = req.body;

    // Validasi input
    if (!username || !password) {
      return res.status(400).json({ error: 'Username dan password wajib diisi' });
    }
```

```
    // Cari user di database
    const user = await pool.query(
      'SELECT * FROM users WHERE username = $1',
      [username]
    );

    if (user.rows.length === 0) {
      return res.status(400).json({ error: 'Username atau password salah' });
    }

    // Verifikasi password
    const validPassword = await bcrypt.compare(
      password,
      user.rows[0].password
    );

    if (!validPassword) {
      return res.status(400).json({ error: 'Username atau password salah' });
    }

    // Generate JWT token
    const token = jwt.sign(
      { id: user.rows[0].id, role: user.rows[0].role },
      process.env.JWT_SECRET,
      { expiresIn: '24h' }
    );

    res.json({
      message: 'Login berhasil',
      token,
      user: {
        id: user.rows[0].id,
        username: user.rows[0].username,
        role: user.rows[0].role
      }
    });
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'Server error' });
  }
});

module.exports = router;
```

Penjelasan: File ini berisi dua endpoint:

- `/register`: Mendaftarkan user baru dengan password yang dienkripsi
- `/login`: Memverifikasi credentials dan menghasilkan JWT token

Catatan untuk peninjau: Screenshot file `routes/auth.js` di VS Code diperlukan di bagian ini. [PERLU SCREENSHOT: File `routes/auth.js` di VS Code dengan syntax highlighting]

3.2 Integrasi Routes ke Server

Langkah 1: Update file `server.js` untuk mengintegrasikan routes:

```
const express = require('express');
const cors = require('cors');
require('dotenv').config();

const authRoutes = require('./routes/auth');

const app = express();
const PORT = process.env.PORT || 5000;

// Middleware
app.use(cors());
app.use(express.json());

// Routes
app.get('/', (req, res) => {
  res.json({ message: 'Kasir API is running' });
});

app.use('/api/auth', authRoutes);

// Start server
app.listen(PORT, () => {
  console.log(`Server is running on port ${PORT}`);
});
```

Langkah 2: Restart server (nodemon akan otomatis restart)

Catatan untuk peninjau: Screenshot file `server.js` yang sudah diupdate diperlukan di bagian ini. [PERLU SCREENSHOT: File `server.js` dengan import authRoutes]

3.3 Testing Endpoint Registrasi dan Login

Langkah 1: Buka Postman atau gunakan curl untuk testing

Langkah 2: Test endpoint registrasi:

```
curl -X POST http://localhost:5000/api/auth/register \
  -H "Content-Type: application/json" \
  -d '{"username":"admin","password":"admin123","role":"admin"}'
```

Anda harus melihat response:

```
{
  "message": "User berhasil didaftarkan",
  "user": {
    "id": 1,
    "username": "admin",
    "role": "admin"
  }
}
```

Langkah 3: Test endpoint login:

```
curl -X POST http://localhost:5000/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username":"admin","password":"admin123"}'
```

Anda harus melihat response dengan JWT token:

```
{
  "message": "Login berhasil",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": 1,
    "username": "admin",
    "role": "admin"
  }
}
```

Catatan untuk peninjau: Screenshot Postman atau terminal dengan response dari endpoint registrasi dan login diperlukan di bagian ini. [PERLU SCREENSHOT: Response dari endpoint /register dan /login]

Bab 4: Implementasi CRUD Produk

4.1 Endpoint CRUD Produk

Langkah 1: Buat file `routes/products.js` :

```
const express = require('express');
const router = express.Router();
const pool = require('../db');

// Dapatkan semua produk
router.get('/', async (req, res) => {
  try {
    const products = await pool.query(
      'SELECT * FROM products ORDER BY created_at DESC'
    );
    res.json(products.rows);
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'Server error' });
  }
});

// Dapatkan satu produk
router.get('/:id', async (req, res) => {
  try {
    const { id } = req.params;
    const product = await pool.query(
      'SELECT * FROM products WHERE id = $1',
      [id]
    );

    if (product.rows.length === 0) {
      return res.status(404).json({ error: 'Produk tidak ditemukan' });
    }

    res.json(product.rows[0]);
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'Server error' });
  }
});

// Tambah produk baru
router.post('/', async (req, res) => {
  try {
    const { name, price, stock, category } = req.body;

    if (!name || !price) {
      return res.status(400).json({ error: 'Nama dan harga wajib diisi' });
    }

    const newProduct = await pool.query(
        'INSERT INTO products (name, price, stock, category) VALUES ($1, $2, $3, $4)
RETURNING *',
      [name, price, stock || 0, category || '']
    );

    res.status(201).json(newProduct.rows[0]);
  } catch (err) {
    console.error(err);
```

```
    res.status(500).json({ error: 'Server error' });
  }
});

// Update produk
router.put('/:id', async (req, res) => {
  try {
    const { id } = req.params;
    const { name, price, stock, category } = req.body;

    const updatedProduct = await pool.query(
      'UPDATE products SET name = $1, price = $2, stock = $3, category = $4, updated_at =
CURRENT_TIMESTAMP WHERE id = $5 RETURNING *',
      [name, price, stock, category, id]
    );

    if (updatedProduct.rows.length === 0) {
      return res.status(404).json({ error: 'Produk tidak ditemukan' });
    }

    res.json(updatedProduct.rows[0]);
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'Server error' });
  }
});

// Hapus produk
router.delete('/:id', async (req, res) => {
  try {
    const { id } = req.params;
    const deletedProduct = await pool.query(
      'DELETE FROM products WHERE id = $1 RETURNING *',
      [id]
    );

    if (deletedProduct.rows.length === 0) {
      return res.status(404).json({ error: 'Produk tidak ditemukan' });
    }

    res.json({ message: 'Produk berhasil dihapus' });
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'Server error' });
  }
});

module.exports = router;
```

Penjelasan: File ini berisi 5 endpoint CRUD:

- `GET /products` : Dapatkan semua produk
- `GET /products/:id` : Dapatkan satu produk

- `POST /products` : Tambah produk baru
- `PUT /products/:id` : Update produk
- `DELETE /products/:id` : Hapus produk

Catatan untuk peninjau: Screenshot file `routes/products.js` di VS Code diperlukan di bagian ini. [PERLU SCREENSHOT: File `routes/products.js` di VS Code dengan syntax highlighting]

4.2 Integrasi Routes Produk ke Server

Langkah 1: Update file `server.js` :

```
const express = require('express');
const cors = require('cors');
require('dotenv').config();

const authRoutes = require('./routes/auth');
const productRoutes = require('./routes/products');

const app = express();
const PORT = process.env.PORT || 5000;

// Middleware
app.use(cors());
app.use(express.json());

// Routes
app.get('/', (req, res) => {
  res.json({ message: 'Kasir API is running' });
});

app.use('/api/auth', authRoutes);
app.use('/api/products', productRoutes);

// Start server
app.listen(PORT, () => {
  console.log(`Server is running on port ${PORT}`);
});
```

Langkah 2: Restart server

Catatan untuk peninjau: Screenshot file `server.js` yang sudah diupdate dengan productRoutes diperlukan di bagian ini. [PERLU SCREENSHOT: File `server.js` dengan import productRoutes]

4.3 Testing Endpoint Produk

Langkah 1: Test tambah produk:

```
curl -X POST http://localhost:5000/api/products \
  -H "Content-Type: application/json" \
  -d '{"name":"Nasi Goreng","price":25000,"stock":50,"category":"Makanan"}'
```

Langkah 2: Test dapatkan semua produk:

```
curl http://localhost:5000/api/products
```

Anda harus melihat array dengan produk yang baru ditambahkan.

Langkah 3: Test update produk:

```
curl -X PUT http://localhost:5000/api/products/1 \
  -H "Content-Type: application/json" \
  -d '{"name":"Nasi Goreng Spesial","price":28000,"stock":45,"category":"Makanan"}'
```

Langkah 4: Test hapus produk:

```
curl -X DELETE http://localhost:5000/api/products/1
```

Catatan untuk peninjau: Screenshot terminal dengan response dari semua endpoint produk diperlukan di bagian ini. [PERLU SCREENSHOT: Response dari endpoint POST, GET, PUT, dan DELETE untuk produk]

Bab 5: Membangun Frontend React

5.1 Inisialisasi Proyek React

Langkah 1: Buat folder `frontend` di root proyek:

```
npx create-react-app frontend
cd frontend
```

Langkah 2: Instal dependensi tambahan:

```
npm install axios react-router-dom tailwindcss
npx tailwindcss init -p
```

Langkah 3: Konfigurasi Tailwind CSS di `frontend/tailwind.config.js`:

```
module.exports = {
  content: [
    "./src/**/*.{js,jsx,ts,tsx}",
  ],
  theme: {
    extend: {},
  },
  plugins: [],
}
```

Langkah 4: Tambahkan Tailwind directives di `frontend/src/index.css`:

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

Langkah 5: Jalankan React development server:

```
npm start
```

Browser akan otomatis membuka http://localhost:3000 dengan halaman default React.

Catatan untuk peninjau: Screenshot browser dengan halaman default React diperlukan di bagian ini. [PERLU SCREENSHOT: Browser menampilkan halaman default React di localhost:3000]

5.2 Membuat Layout dan Routing

Langkah 1: Buat folder `components` di `frontend/src`:

```
mkdir src/components
```

Langkah 2: Buat file `src/components/Layout.js`:

```
import { Link, Outlet } from 'react-router-dom';

function Layout() {
  return (
    <div className="min-h-screen bg-gray-100">
      <nav className="bg-blue-600 text-white p-4">
        <div className="container mx-auto flex justify-between items-center">
          <h1 className="text-2xl font-bold">Aplikasi Kasir</h1>
          <div className="space-x-4">
            <Link to="/" className="hover:underline">Dashboard</Link>
            <Link to="/products" className="hover:underline">Produk</Link>
            <Link to="/transactions" className="hover:underline">Transaksi</Link>
          </div>
        </div>
      </nav>
      <main className="container mx-auto p-6">
        <Outlet />
      </main>
    </div>
  );
}

export default Layout;
```

Penjelasan: Layout ini berisi navbar dengan navigasi dan area konten yang akan diisi oleh child routes.

Catatan untuk peninjau: Screenshot file `Layout.js` di VS Code diperlukan di bagian ini. [PERLU SCREENSHOT: File `Layout.js` di VS Code dengan syntax highlighting]

5.3 Membuat Halaman Dashboard

Langkah 1: Buat folder `pages` di `frontend/src`:

```
mkdir src/pages
```

Langkah 2: Buat file `src/pages/Dashboard.js`:

```
import { useState, useEffect } from 'react';
import axios from 'axios';

function Dashboard() {
  const [stats, setStats] = useState({
    totalProducts: 0,
    totalTransactions: 0,
    totalRevenue: 0
  });

  useEffect(() => {
    // Akan diimplementasikan: API call untuk mendapatkan statistik dashboard
    console.log('Loading dashboard stats...');
  }, []);

  return (
    <div>
      <h2 className="text-3xl font-bold mb-6">Dashboard</h2>
      <div className="grid grid-cols-1 md:grid-cols-3 gap-6">
        <div className="bg-white p-6 rounded-lg shadow">
          <h3 className="text-gray-500 text-sm">Total Produk</h3>
          <p className="text-3xl font-bold text-blue-600">{stats.totalProducts}</p>
        </div>
        <div className="bg-white p-6 rounded-lg shadow">
          <h3 className="text-gray-500 text-sm">Total Transaksi</h3>
          <p className="text-3xl font-bold text-green-600">{stats.totalTransactions}</p>
        </div>
        <div className="bg-white p-6 rounded-lg shadow">
          <h3 className="text-gray-500 text-sm">Total Pendapatan</h3>
          <p className="text-3xl font-bold text-purple-600">
            Rp {stats.totalRevenue.toLocaleString('id-ID')}
          </p>
        </div>
      </div>
    </div>
  );
}

export default Dashboard;
```

Catatan untuk peninjau: Screenshot file `Dashboard.js` di VS Code diperlukan di bagian ini. [PERLU SCREENSHOT: File `Dashboard.js` di VS Code dengan syntax highlighting]

5.4 Membuat Halaman Produk

Langkah 1: Buat file `src/pages/Products.js` :

```
import { useState, useEffect } from 'react';
import axios from 'axios';

function Products() {
  const [products, setProducts] = useState([]);
  const [showForm, setShowForm] = useState(false);
  const [formData, setFormData] = useState({
    name: '',
    price: '',
    stock: '',
    category: ''
  });

  useEffect(() => {
    loadProducts();
  }, []);

  const loadProducts = async () => {
    try {
      const response = await axios.get('http://localhost:5000/api/products');
      setProducts(response.data);
    } catch (err) {
      console.error('Error loading products:', err);
    }
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    try {
      await axios.post('http://localhost:5000/api/products', formData);
      setFormData({ name: '', price: '', stock: '', category: '' });
      setShowForm(false);
      loadProducts();
    } catch (err) {
      console.error('Error adding product:', err);
    }
  };

  return (
    <div>
      <div className="flex justify-between items-center mb-6">
        <h2 className="text-3xl font-bold">Produk</h2>
        <button
          onClick={() => setShowForm(!showForm)}
          className="bg-blue-600 text-white px-4 py-2 rounded hover:bg-blue-700"
        >
          {showForm ? 'Batal' : 'Tambah Produk'}
        </button>
      </div>

      {showForm && (
        <form onSubmit={handleSubmit} className="bg-white p-6 rounded-lg shadow mb-6">
          <div className="grid grid-cols-2 gap-4">
            <div>
              <label className="block text-sm font-medium mb-1">Nama Produk</label>
```

```
              <input
                type="text"
                value={formData.name}
                onChange={(e) => setFormData({...formData, name: e.target.value})}
                className="w-full border rounded px-3 py-2"
                required
              />
            </div>
            <div>
              <label className="block text-sm font-medium mb-1">Harga</label>
              <input
                type="number"
                value={formData.price}
                onChange={(e) => setFormData({...formData, price: e.target.value})}
                className="w-full border rounded px-3 py-2"
                required
              />
            </div>
            <div>
              <label className="block text-sm font-medium mb-1">Stok</label>
              <input
                type="number"
                value={formData.stock}
                onChange={(e) => setFormData({...formData, stock: e.target.value})}
                className="w-full border rounded px-3 py-2"
              />
            </div>
            <div>
              <label className="block text-sm font-medium mb-1">Kategori</label>
              <input
                type="text"
                value={formData.category}
                onChange={(e) => setFormData({...formData, category: e.target.value})}
                className="w-full border rounded px-3 py-2"
              />
            </div>
          </div>
          <button
            type="submit"
            className="mt-4 bg-green-600 text-white px-6 py-2 rounded hover:bg-green-700"
          >
            Simpan
          </button>
        </form>
      )}

      <div className="bg-white rounded-lg shadow overflow-hidden">
        <table className="w-full">
          <thead className="bg-gray-50">
            <tr>
                    <th  className="px-6  py-3  text-left  text-xs  font-medium  text-gray-500
uppercase">Nama</th>
                    <th  className="px-6  py-3  text-left  text-xs  font-medium  text-gray-500
uppercase">Harga</th>
                    <th  className="px-6  py-3  text-left  text-xs  font-medium  text-gray-500
```

```
uppercase">Stok</th>
                  <th className="px-6 py-3 text-left text-xs font-medium text-gray-500
uppercase">Kategori</th>
            </tr>
          </thead>
          <tbody className="divide-y divide-gray-200">
            {products.map((product) => (
              <tr key={product.id}>
                <td className="px-6 py-4">{product.name}</td>
                <td className="px-6 py-4">Rp {product.price.toLocaleString('id-ID')}</td>
                <td className="px-6 py-4">{product.stock}</td>
                <td className="px-6 py-4">{product.category}</td>
              </tr>
            ))}
          </tbody>
        </table>
      </div>
    </div>
  );
}

export default Products;
```

Catatan untuk peninjau: Screenshot file `Products.js` di VS Code diperlukan di bagian ini. [PERLU SCREENSHOT: File `Products.js` di VS Code dengan syntax highlighting]

5.5 Konfigurasi Routing

Langkah 1: Update file `src/App.js`:

```
import { BrowserRouter, Routes, Route } from 'react-router-dom';
import Layout from './components/Layout';
import Dashboard from './pages/Dashboard';
import Products from './pages/Products';

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Layout />}>
          <Route index element={<Dashboard />} />
          <Route path="products" element={<Products />} />
        </Route>
      </Routes>
    </BrowserRouter>
  );
}

export default App;
```

Langkah 2: Buka browser di http://localhost:3000. Anda harus melihat halaman Dashboard dengan navbar.

Catatan untuk peninjau: Screenshot browser dengan halaman Dashboard dan navbar diperlukan di bagian ini. [PERLU SCREENSHOT: Browser menampilkan halaman Dashboard dengan navbar biru]

Bab 6: Deployment ke Vercel

6.1 Persiapan untuk Deployment

Langkah 1: Buat file `vercel.json` di root proyek backend:

```
{
  "version": 2,
  "builds": [
    {
      "src": "server.js",
      "use": "@vercel/node"
    }
  ],
  "routes": [
    {
      "src": "/(.*)",
      "dest": "server.js"
    }
  ]
}
```

Langkah 2: Push kode ke GitHub:

```
git init
git add .
git commit -m "Initial commit"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/kasir-app.git
git push -u origin main
```

Catatan untuk peninjau: Screenshot terminal dengan output git push diperlukan di bagian ini. [PERLU SCREENSHOT: Terminal dengan output git push ke GitHub]

6.2 Deploy Backend ke Vercel

Langkah 1: Login ke Vercel dan klik "New Project"

Langkah 2: Import repository GitHub Anda

Langkah 3: Konfigurasi environment variables di Vercel:

- `DATABASE_URL`: Connection string PostgreSQL (gunakan database cloud seperti Supabase atau Neon)
- `JWT_SECRET`: Random string untuk JWT

Langkah 4: Klik "Deploy" dan tunggu proses deployment selesai

Langkah 5: Setelah deployment selesai, Anda akan mendapat URL seperti `https://kasir-app.vercel.app`

Catatan untuk peninjau: Screenshot dashboard Vercel dengan project yang sedang di-deploy diperlukan di bagian ini. [PERLU SCREENSHOT: Dashboard Vercel dengan project kasir-app dan status deployment]

6.3 Deploy Frontend ke Vercel

Langkah 1: Update `frontend/src/App.js` untuk menggunakan URL backend production:

```
const API_URL = process.env.REACT_APP_API_URL || 'http://localhost:5000/api';
```

Langkah 2: Buat file `frontend/.env.production`:

```
REACT_APP_API_URL=https://kasir-app.vercel.app/api
```

Langkah 3: Push perubahan ke GitHub

Langkah 4: Di Vercel, buat project baru untuk frontend

Langkah 5: Set root directory ke `frontend` dan deploy

Langkah 6: Setelah deployment selesai, Anda akan mendapat URL seperti `https://kasir-frontend.vercel.app`

Catatan untuk peninjau: Screenshot browser dengan aplikasi kasir yang sudah live di Vercel diperlukan di bagian ini. [PERLU SCREENSHOT: Browser menampilkan aplikasi kasir yang sudah live di URL Vercel]

Penutup

Selamat! Anda telah berhasil membangun aplikasi kasir dari nol menggunakan vibecoding. Aplikasi ini memiliki:

- Authentication dengan JWT
- CRUD produk
- Frontend React dengan Tailwind CSS
- Deployment ke Vercel

Langkah selanjutnya:

1. Tambahkan fitur transaksi dan laporan penjualan
2. Implementasi authorization berdasarkan role

3. Tambahkan validasi input yang lebih ketat
4. Implementasi testing otomatis
5. Monitor performa dan error

Catatan untuk peninjau: Screenshot aplikasi kasir final dengan semua fitur yang sudah diimplementasikan diperlukan di bagian ini. [PERLU SCREENSHOT: Aplikasi kasir final dengan dashboard, produk, transaksi, dan laporan]

Versi: 1.0
Tanggal: Oktober 2026
Penyusun: Siraj Nur Ihrom, Founder & Lead Developer SNISHOP
Standar: SNISHOP-PUBLISHING v2.0
Profil: Tutorial Step-by-Step
Status: Draft untuk tinjauan; screenshot dan testing lebih lanjut masih dalam proses